



ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

SCHOOL OF ELECTRICAL ENGINEERING AND COMPUTING

Integrated Engineering Team Project

Software Requirement Specification Document

Project Name: SafeMesh Offline-First Decentralized Communication Network

Author(s): SafeMesh Development Team

Table of Contents	Page
1. Introduction	4
1.1 Purpose of this Document	4
1.2 Document Scope and Objectives	4
1.3 Intended Audience	4
1.4 References and Standards.....	5
2. Overall Description	5
2.1 Product Vision and Mission.....	5
2.2 Business Context and Problem Statement.....	5
2.3 Product Perspective.....	5
2.4 User Classes and Characteristics.....	6
2.5 Operating Environment.....	6
2.6 Design and Implementation Constraints.....	6
2.7 Assumptions and Dependencies.....	6
3. Functional Requirements	7
3.1 Node Initialization and Provisioning.....	7
3.2 Message Composition and Transmission.....	7
3.3 Smart Relaying and Deduplication.....	7
3.4 SOS Emergency Broadcast.....	7

3.5 Gateway Bridging and Dashboard Integration.....	7
3.6 Network Key Management and Encryption.....	7
3.7 Battery and Solar Power Management.....	8
3.8 Network Monitoring and Visualization.....	8
4. Non-Functional Requirements	8
5. External Interfaces	8
6. Other Requirements	9
6.1 Data Retention, Privacy and Compliance.....	9
6.2 Analytics and Reporting.....	9
6.3 Support and Maintenance Requirements.....	9
6.4 Future Phases (Out of MVP Scope).....	9
7. Glossary	10
8.Conclusion	11
9.Reference	12

1. Introduction

1.1 Purpose of this Document

This Software Requirements Specification (SRS) provides a complete, unambiguous description of what SafeMesh must do, how it should behave, and the standards it must meet. It is the foundational document that will guide firmware development, hardware integration, dashboard development, testing, investor discussions, grant applications, and real-world field deployments.

1.2 Document Scope and Objectives

The document captures all functional and non-functional requirements for the Minimum Viable Product (MVP) — a fully functional offline mesh network consisting of ESP32 nodes and a web-based monitoring dashboard. The system is targeted for deployment in off-grid rural Ethiopia and similar environments in 2026.

1.3 Intended Audience

- Founders and project team
- Embedded systems and firmware developers
- Full-stack dashboard developers
- Hardware engineers and field deployment officers
- Investors, donors, and accelerator programs
- University researchers (e.g., ASTU) and government stakeholders
- NGOs working in disaster resilience and rural connectivity

1.4 References and Standards

- ✚ Espressif ESP-NOW User Guide and API Reference
- ✚ MicroPython Documentation (ESP32 Port)
- ✚ IEEE Recommended Practice for Software Requirements Specifications (IEEE 830-1998)
- ✚ Ethiopian National Digital Transformation Strategy 2025
- ✚ GSMA Mobile for Development – Off-grid Connectivity Reports
- ✚ AES-128 Encryption Standards (FIPS PUB 197)

2. Overall Description

2.1 Product Vision and Mission

Vision: By 2030, SafeMesh will become the most accessible, ultra-low-cost decentralized communication solution across Africa, significantly improving safety and coordination in off-grid communities.

Mission: Provide reliable, solar-powered, zero-operating-cost text messaging and SOS services in areas without cellular coverage using affordable ESP32-based mesh nodes.

2.2 Business Context and Problem Statement

In developing regions, deep rural areas and difficult terrains often lack reliable cellular coverage due to high infrastructure costs. Private organizations such as construction crews, agricultural farms, and university security teams incur constant airtime expenses. SafeMesh addresses this critical gap by offering a deployable, independent, and secure offline communication network.

2.3 Product Perspective

SafeMesh is a new hybrid system combining low-power embedded hardware with a web monitoring dashboard. It competes with expensive VHF radios and satellite solutions by delivering dramatically lower cost and full independence from cellular infrastructure.

2.4 User Classes and Characteristics

- ✚ **Field Users** (farmers, rangers, security personnel): Low-to-medium technical literacy; interact via physical buttons, LEDs, or basic OLED.
- ✚ **Network Administrators**: Manage keys and monitor the system via web dashboard.
- ✚ **Deployers / Technicians**: Handle initial flashing and physical deployment.
- ✚ **Monitors**: View messages and alerts through the centralized dashboard.

2.5 Operating Environment

- Firmware: MicroPython on ESP32 microcontrollers
- Backend: Python + Flask with serial bridging
- Frontend: React.js Progressive Web App (PWA)
- Radio: 2.4 GHz ESP-NOW protocol
- Power: Solar + 18650 Li-ion battery
- Environment: Rugged outdoor conditions (-10°C to +60°C)

2.6 Design and Implementation Constraints

- ✚ Must operate completely offline using only radio communication
- ✚ Packet size limited to ESP-NOW constraints (≤ 250 bytes)
- ✚ Extremely low power consumption for long-term solar operation
- ✚ Total node cost target under \$8 USD
- ✚ Strong security using AES-128 encryption

2.7 Assumptions and Dependencies

- ESP32 modules and 18650 batteries are locally available or importable
- Basic technical support available for initial node configuration
- Nodes will be placed within effective radio range (1–5 km per hop)
- A host computer (laptop or Raspberry Pi) is available for the gateway

3. Functional Requirements (MVP)

3.1 Node Initialization and Provisioning

Nodes must support easy initial setup. On first boot, the node accepts a Network Key via serial or button combination and stores it securely in Non-Volatile Storage (NVS).

3.2 Message Composition and Transmission

Users shall be able to compose short text messages and send them. The system generates a unique Sequence ID, encrypts the message using AES-128, and broadcasts it via ESP-NOW.

3.3 Smart Relaying and Deduplication

Upon receiving a message, nodes check the Sequence ID. New messages are relayed; duplicate messages are discarded to prevent broadcast storms.

3.4 SOS Emergency Broadcast

A dedicated long-press mechanism triggers high-priority SOS messages, optionally including GPS coordinates, with repeated flooding for maximum reach.

3.5 Gateway Bridging and Dashboard Integration

The Gateway Node forwards all valid messages to the Flask backend via UART serial communication for storage and real-time visualization on the React dashboard.

3.6 Network Key Management and Encryption

Administrators can create, rotate, and distribute Network Keys. All radio traffic must be encrypted with AES-128 using the shared private key.

3.7 Battery and Solar Power Management

Nodes shall monitor battery level and solar charging status and periodically broadcast telemetry data. Aggressive deep-sleep cycles shall be implemented to maximize operational time.

3.8 Network Monitoring and Visualization

The web dashboard shall display live messages, SOS alerts with location, node status, battery levels, and basic network activity.

4. Non-Functional Requirements

- ✚ **Performance:** Message propagation latency < 10 seconds per hop
- ✚ **Reliability:** Minimum 95% delivery success rate in 5-hop topology
- ✚ **Power Efficiency:** Average current consumption < 100 μ A in normal operation
- ✚ **Security:** Full AES-128 encryption with replay attack protection
- ✚ **Usability:** Field users able to send messages and SOS with minimal training
- ✚ **Scalability:** Support up to 200 nodes in a single mesh network
- ✚ **Maintainability:** Modular firmware design with easy serial updates
- ✚ **Offline Capability:** Full mesh functionality without internet connection

5. External Interfaces

- ✓ ESP-NOW Peer-to-Peer Radio Protocol
- ✓ UART Serial Communication (Gateway to Host)
- ✓ Flask RESTful APIs (Backend to Frontend)
- ✓ React.js Web Dashboard (PWA)
- ✓ Optional NEO-6M GPS Module (UART)

6. Other Requirements

6.1 Data Retention, Privacy and Compliance

Messages stored on the gateway for a configurable period (default 30 days). Private network keys ensure data privacy. The system shall comply with Ethiopian data protection regulations.

6.2 Analytics and Reporting

The dashboard shall generate reports on message volume, SOS incidents, node uptime, and network health for project evaluation and donor reporting.

6.3 Support and Maintenance Requirements

Comprehensive deployment manual, field troubleshooting guide, and over-the-serial firmware update capability. Open-source documentation for community support.

6.4 Future Phases (Out of MVP Scope)

- Voice messaging capability
- Native Android companion application
- Integration with LoRa for extended range
- AI-based network optimization
- Hybrid cellular failover mode

7. Glossary

ESP-NOW – Espressif’s low-power peer-to-peer wireless protocol **MVP** – Minimum Viable Product **PWA** – Progressive Web App **Controlled Flooding** – Broadcast relaying with sequence ID deduplication **Gateway Node** – ESP32 node connected to a computer via serial **Sequence ID** – Unique identifier used to prevent message loops **AES-128** – Advanced Encryption Standard (128-bit)

8. Conclusion

SafeMesh presents an innovative, cost-effective, and reliable solution to the critical challenge of communication in off-grid and remote areas. By combining low-cost ESP32 hardware, ESP-NOW protocol, and intelligent mesh networking, the system delivers essential text messaging and SOS capabilities without dependency on cellular infrastructure or recurring costs.

This Software Requirements Specification provides a comprehensive, implementation-ready blueprint covering all functional, non-functional, architectural, and security requirements. The design emphasizes ultra-low power consumption, strong encryption, ease of deployment and real-world practicality for rural communities, national parks, construction sites, and university campuses.

SafeMesh has the potential to significantly improve safety, coordination, and connectivity in underserved regions while demonstrating the power of accessible and sustainable technology. The successful implementation of this system will mark a meaningful step toward digital inclusion and resilience in Ethiopia and beyond.

Sources / References (Additional Section as requested)

- Espressif Systems. (2024). *ESP-NOW User Guide*.
- MicroPython Documentation. (2025). *ESP32 Quick Reference*.
- IEEE Std 830-1998. *Recommended Practice for Software Requirements Specifications*.
- GSMA. (2024–2025). *Mobile for Development – Off-grid Connectivity in Africa*.
- Ethiopian Ministry of Innovation and Technology. *National Digital Strategy 2025*.
- Federal Democratic Republic of Ethiopia. *Personal Data Protection Proclamation (Draft)*.